NightShift: Batch LLM Inference on Charging Smartphones

Anonymous Author(s)

ABSTRACT

Many smartphones spend the night plugged in, on Wi-Fi and idle. We ask whether such a pool can run inference for a large language model too big for any single phone, and what it would take. A 70B-parameter model at 4-bit needs about 40 GB, so one copy is a 16-stage pipeline of phones that leave whenever their owners unplug them. Using a simulator whose phone and network parameters are explicit, replaceable assumptions, we find three things. First, the pool is a night-time resource: 98% of a day's tokens are produced between 22:00 and 08:00. Second, each output token takes seconds (2.3 s at 60 ms per hop), so the useful workload is batch work, not chat. Third, once phones come and go, the main cost of churn is moving model shards to the phones that replace leavers, not recomputing lost work: even a cache-aware baseline downloads 363 GB per million tokens, about 4.0 GB per serving phone per day. NightShift gives every phone a permanent home block, lets a vacant stage wait briefly for a phone that already holds it, and consolidates broken pipelines. With 1,000 phones it downloads 6–7× less data than every baseline we compare, with throughput equal to Petals-style replay within noise and 5% below the best baseline; at 4,000 phones it is ahead on both (+7% throughput against consolidation, 10× less traffic than cache-aware). Keeping warm standbys, the obvious remedy, cuts throughput by 39%. We release the simulator and a benchmark kit so that the assumed parameters can be replaced with measurements.

CCS CONCEPTS

• **Computer systems organization** → **Distributed architectures**; • **Human-centered computing** → *Ubiquitous and mobile computing*; • **Computing methodologies** → *Machine learning*.

KEYWORDS

large language models, distributed inference, smartphones, volunteer computing, pipeline parallelism, churn

1 INTRODUCTION

Volunteer computing has a long record: BOINC and SETI@home turned idle desktops into a scientific computer [2], and Google's federated learning system trains models on phones only while they are "idle, charging and on an unmetered network" [4]. Large language models (LLMs) are the largest compute consumer of the decade, which invites the question: could the idle phones of the world run them?

The question has an easy wrong answer in each direction. "Yes, a billion phones beat any data centre" ignores that generation is sequential and that a large model does not fit on a phone. "No, phones are too small" ignores that the same was said of volunteer desktops, and that systems such as Petals [5, 6] already serve 100B-class models from pipelines of volunteer GPUs over the internet. The interesting question is quantitative: *for which workloads, at what throughput and cost, and which design choices matter?*

This paper makes three contributions.

(1) A first-principles analysis (§2) of what a phone pool can and cannot do: it suits throughput-oriented batch jobs, not interactive chat, and it reads far more weight memory per token than a GPU, so its case rests on marginal cost, not efficiency.
(2) A churn-aware simulator (§5) driven by a diurnal model of when phones are charging and on Wi-Fi. Every phone, network and user parameter is explicit and can be replaced by a measurement; we release a benchmark kit that produces those measurements on Android phones.
(3) NightShift (§4), a placement and recovery design built on one observation from the simulator: once phones come and go, moving model shards costs more than redoing computation. A permanent home block per phone, a short wait for a phone that holds the vacant block, and consolidation of broken pipelines cut data downloaded 6–7× against every baseline at 1,000 phones, including a fair cache-aware one, and 5× even when phones cache the whole model. Throughput matches replay within noise and is 5% below the best baseline at this size; NightShift's lead grows with the pool, and it trails at 500 phones. An ablation shows that neither the wait nor consolidation helps alone, and that dedicated standbys make things worse.

We are explicit about what is and is not measured. All results here come from simulation with stated assumptions (Table 1); §7 describes the measurements that will replace them.

2 THE PHYSICS OF A PHONE POOL

Memory. A 70B model quantised to about 4 bits per weight occupies roughly 40 GB; we model 80 layers of 0.5 GB. An Android app running in the background can rarely hold more than a fraction of a phone's RAM; we assume 45%, so an 8 GB phone offers 3.6 GB. With the key-value (KV) cache for a batch of 32 sequences of 768 tokens, a five-layer block needs 3.0 GB, so a model copy is a pipeline of 16 phones, and phones with 6 GB of RAM cannot take part at all (90% of our pool is eligible).

Latency. Generating one token visits all 16 stages in order. At roughly 15 ms per layer on a mid-range phone and 60 ms per wide-area hop, a token takes about 1.2 s of compute plus 1.0 s of network, i.e. around two seconds. Neither speculative decoding [16] nor faster phones change the order of magnitude while the hops remain. A conversational assistant is therefore out of reach over the internet; a household network (5 ms hops) brings latency to 2.0 s.

Throughput. Latency need not limit throughput. With many sequences in flight, a pipeline keeps every stage busy, as in GPipe-style micro-batching [11] and continuous batching servers [14, 31]. Batch workloads such as offline evaluation, synthetic data generation, document summarisation or embedding a corpus tolerate hours of latency and are a large share of LLM demand, the same regime FlexGen targets on a single GPU [25].

Memory traffic and energy. The price is efficiency. A data-centre GPU serving a batch of 128 reads each weight once per 128 tokens, about 0.3 GB of weight traffic per token for a 70B model. A phone pipeline with 32 sequences over 16 stages processes micro-batches of two, so it reads about 20 GB per token, roughly 60× more. At an assumed 4 W per active phone and 1 W while waiting, the simulated pool spends about 6 J per token; we have not measured a GPU server on the same workload, and that comparison is part of the validation plan. The phone pool's argument is therefore not efficiency but *marginal cost*: the hardware is already bought, already powered at night and otherwise idle. Whether that argument survives battery wear and the owner's electricity bill is a question for measurement and incentive design, which we leave open (§8).

3 RELATED WORK

Volunteer and federated computing. BOINC [2] established redundant execution and work units for untrusted volunteers but ran independent jobs. Federated learning [4, 20] runs on phones at production scale and established the eligibility rule we adopt (charging, Wi-Fi, idle); each phone trains a full small model, and nothing is split across devices. Device availability in federated learning has been measured at large scale [15, 29]; those traces are natural inputs to our simulator.

LLMs over unreliable consumer hardware. Petals [5, 6] serves large models from pipelines of volunteer GPUs and recovers from departures by routing around them and replaying the lost stage's cache; it is the baseline we call *replay*. SWARM [23], Learning@home [24] and DiLoCo [7] train over unreliable or poorly connected nodes. Deserve [27] studies decentralised serving on volunteer GPUs, and HexGen [13] schedules LLM inference over heterogeneous, decentralised GPUs. SpotServe [21] serves LLMs on preemptible cloud instances, migrating context when an instance is reclaimed. For training, Bamboo [26] adds redundant computation and Oobleck [12] keeps redundant pipeline templates to survive failures, and Parcae [8] predicts spot-instance availability to adapt parallelism proactively, the same idea as our lifetime grouping. Those nodes are GPUs with tens of gigabytes and, often, minutes of warning; ours are phones with a few gigabytes and no warning, and moving a shard to a replacement costs the volunteer data.

Splitting models across nearby devices. LinguaLinked [33] runs one LLM across several Android phones, Galaxy [30] and EdgeShard [32] split transformers across edge devices, and exo [9] does so for a household's laptops and phones; prima.cpp [17] runs 70B-class models on a home cluster of everyday devices; earlier, MoDNN [19] and DeepThings [34] split CNNs, and CWC [3] scheduled jobs on the idle, charging phones of an organisation. These assume a small, stable set of devices. We study the opposite regime: a large pool in which membership changes all the time.

Single-phone inference. LLM in a flash [1], PowerInfer-2 [28] and runtimes such as llama.cpp [10] make single phones faster, and MobileLLM [18] designs models that fit one phone. These are complementary: a faster phone shortens every stage, and a model that fits one phone needs no pipeline at all, which is the right choice whenever such a model is good enough.

4 NIGHTSHIFT

A coordinator holds a job queue and a fixed *shard catalogue*: the model cut into 16 blocks of 5 layers. A phone joins the pool when it is charging, on Wi-Fi and idle, and leaves without warning. A pipeline is one phone per block; it advances a batch of 32 sequences in lock step. When a member leaves, its sequences pause until the stage is refilled. NightShift has three parts.

Sticky shards. Every phone is assigned a *home block* once and keeps it in flash storage indefinitely. A pipeline stage is only ever filled by a phone whose home block matches; loading from flash takes seconds, whereas downloading 2.5 GB over a residential link takes minutes and costs the owner data. If no matching phone is idle, the vacant stage waits up to five minutes before falling back to a phone that must download the block; a pipeline whose stage has been vacant for ten minutes is dissolved and its phones return to the pool (this rule applies to every policy). Because the catalogue is fixed, the coordinator only needs to keep the 16 home-block populations balanced.

Lifetime-grouped pipelines. A pipeline lives only as long as its shortest-lived member. The coordinator predicts when each phone's charging session ends, from that user's history of sessions starting at a similar time of day, sorts each block's idle phones by predicted end, and builds the i-th pipeline from the i-th phone of every block. Phones expected to stay until morning end up together, and short daytime sessions are grouped with each other instead of cutting long-lived pipelines short.

Dawn consolidation. In the morning, phones leave faster than idle phones arrive, leaving many pipelines with one or two vacant stages that nobody can fill. Waiting wastes every phone in those pipelines. When at least two pipelines are broken, NightShift dissolves the most damaged one (most vacancies, then least progress) and lends its phones, which already hold blocks of the same catalogue, to the others. A pool that shrinks keeps as many *complete* pipelines as it can afford. Because the donor's phones hold the blocks the survivors are missing, consolidation is also what keeps the wait short: without it, a vacant stage often waits for a home-block phone that is not coming.

Recovery. A replacement phone recomputes the KV cache of its block for the tokens processed so far, as in Petals. To do so it needs the input activations of its block for every token so far (prompt and generated). We assume the previous stage keeps them and sends them over the replacement's downlink, 16 KiB per token per sequence (8,192 hidden dimensions in fp16); this transfer takes time and is counted as traffic for every policy. We compare replay with restarting the batch.

What we chose not to do. Keeping k warm standbys per stage turns a failure into a fast swap but takes $k \times 16$ phones out of service per pipeline. We evaluate it as a baseline.

5 SIMULATION METHODOLOGY

The simulator advances in 10-second ticks. Each phone has a RAM class, a per-layer decode time and a downlink speed drawn from Table 1, and a sequence of charging sessions drawn from the diurnal model, or read from a real trace. A pipeline's *time per output token* is

Table 1: Simulator parameters. All are assumptions until replaced by measurements from the benchmark kit (§7).

Model	80 layers × 0.5 GB, KV 4 KiB/token/layer
Workload	batches of 32 seq., 512 prompt + 256 output tokens
Prefill cost	1/8 of a decode token
Phones	RAM, share of pool, ms per layer: 6 GB, 10%, 30; 8 GB, 40%, 18; 12 GB, 35%, 12; 16 GB, 15%, 10 (log-normal spread σ=0.2)
Usable RAM	45% of physical
Network	60 ms per hop; downlink median 50 Mbit/s
Flash read	1 GB/s
Power	4 W computing, 1 W waiting
Night charging	85% of nights, start 23:00±1 h, 7±1.5 h
Day charging	1.5 sessions/day, median 35 min, 60% on Wi-Fi
Flash cache	1 block (2.5 GB) per phone; §6 varies it to 16
Activations	16 KiB per token per sequence, sent on repair
Timers	home-block wait 300 s; vacant pipeline dissolved after 600 s
Warm-up	2 days run and discarded before measuring

one round: the sum of its stage times plus one hop per stage. With 16 micro-batches in flight, a round is $\max(\text{latency}, 16 \cdot \max_i \text{stage}_i)$ and produces one token for each of the 32 sequences. A micro-batch of b tokens costs $1 + 0.1(b-1)$ times a single token, and prefill costs 1/8 of a decode token.

Steady state. Seven days of synthetic history train the session-end predictor. The simulator then runs two days under the policy being evaluated and discards them, so that flash caches and pipelines reach that policy's own steady state, and finally measures 24 hours from noon. Every phone starts the warm-up holding one block (its home block), so a policy cannot look good merely by inheriting another's caches. Onboarding traffic, a phone's first download, is outside the measured window for every policy.

Policies. We compare ten policies that differ only in placement and recovery. The baselines are *restart* and *replay* (random placement; restart a broken batch, or replay its cache as Petals does); *cache-aware*, a fair version of replay that fills each stage from idle phones already holding its block before falling back to any idle phone (it never waits); *standby* (replay plus one warm standby per stage); *lifetime* (replay with pipelines grouped by predicted session end); and *consolidate* (replay plus consolidation). The NightShift ablation adds one part at a time: *wait* (home blocks and the five-minute wait, random order), *sticky* (plus lifetime grouping), *NightShift without the wait* (sticky plus consolidation, falling back to a download at once) and *NightShift*. The *upper bound* places every available, eligible phone in a complete pipeline instantly and never loses work.

Metrics and statistics. Throughput is output tokens per second per 1,000 phones in the pool. Traffic is gigabytes downloaded per million output tokens: shards plus repair activations. Results are means over 5 seeds with 1,000 phones (sweeps: 3 seeds); differences between policies are paired by seed, so both see the same phones and sessions, and we report them with 95% t confidence intervals.

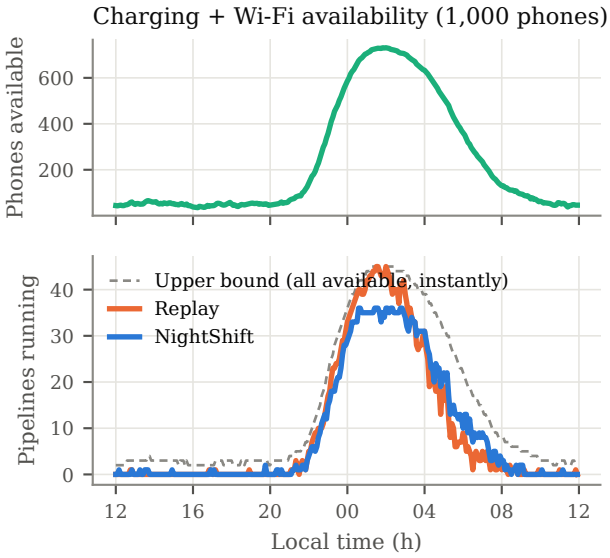


Figure 1: One simulated day. Top: phones available. Bottom: pipelines running and an upper bound that assumes instant, perfect placement.

6 RESULTS

The pool is a night-time machine. Availability swings from 35 to 731 of 1,000 phones over the day (Figure 1), and 98% of tokens are produced between 22:00 and 08:00. Daytime sessions are short and scattered, so few complete 16-phone pipelines form. A phone pool is best treated as a batch system that drains a queue overnight.

Moving shards, not lost work, is the cost of churn. Figure 2 compares the policies. Restart differs from replay by only -4% (±0.4 tokens/s, 95% CI), so little computation is lost to departures. What separates the policies is data movement. Under replay, a repair recruits whichever phone is idle, which rarely holds the needed block: the pool downloads 410 GB per million tokens, of which 362 GB are shards. Preferring phones that already hold the block (cache-aware) helps less than one would expect, to 363 GB, because at a given moment the few idle phones seldom hold the right block; a serving phone still downloads about 4.0 GB a day, enough to make a volunteer's data plan the binding constraint.

NightShift. NightShift downloads 57 GB per million tokens: 7× less than replay, 6× less than cache-aware and 7× less than consolidation; for shards alone the reduction against replay is 12×. A serving phone downloads about 0.7 GB a day instead of 4.4 GB. Its throughput is 109 tokens/s per 1,000 phones against 106 for replay, a paired difference of +2.4 ± 5.1, which is not significant. The best baseline, consolidation, is faster: 115 tokens/s, a paired difference of -6.0 ± 4.0 tokens/s, so NightShift is 5% behind it at this pool size. NightShift suffers 46% fewer stage failures than replay, and its stages spend 83% of their time computing rather than waiting, against 64%. The trade is therefore a few percent of throughput at 1,000 phones for a large cut in the data volunteers must download.

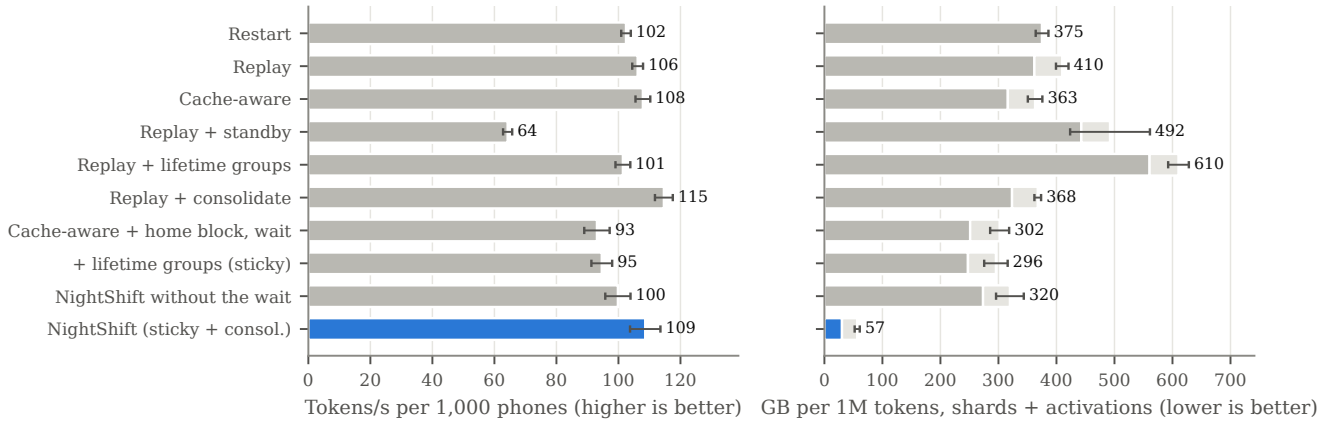


Figure 2: Throughput and data downloaded (shards, dark; repair activations, light) of the ten policies (1,000 phones, mean ± s.d. over 5 seeds). The last four rows add NightShift's parts one at a time.

The ablation: the wait and consolidation need each other. Giving each phone a home block and making vacant stages wait for it (*wait*) cuts traffic only to 302 GB and changes throughput by -13.1 ± 3.5 tokens/s against replay: in the morning, stages wait for home-block phones that have left for the day. Adding lifetime grouping changes little (95 tokens/s, 296 GB). Adding consolidation but no wait (*NightShift without the wait*) recovers some throughput but keeps traffic at 320 GB, because every vacancy still triggers a download. Only together does the design work: consolidation supplies home-block phones from dissolved pipelines, so the wait is usually short and usually rewarded. Lifetime grouping alone, without home blocks, is worse than random (101 tokens/s, 610 GB), because ranking phones by predicted stay overrides any chance of reusing a cached block.

Standbys are a bad trade on phones. One standby per stage cuts throughput by 39%: it removes half of the pool from service to protect pipelines whose expected lifetime, at night, is hours. Redundancy pays off when failures are frequent relative to the cost of idling capacity, as for preemptible cloud GPUs [12, 26], but not here.

Larger caches do not close the gap. One might expect the baselines to catch up if phones simply kept more blocks in flash. Figure 3 (left) varies the cache from one block to all 16 (the whole 40 GB model). Baseline traffic roughly halves (replay 213, cache-aware 207, consolidation 182 GB per million tokens with a full cache) while NightShift falls to 40 GB, still 5× less than cache-aware. Keeping 40 GB on a volunteer's phone is, in any case, a large request; we assume one block.

The wait is a knob. Figure 3 (right) varies how long a vacant stage waits for a home-block phone. With no wait, NightShift is the no-wait ablation above (99 tokens/s, 314 GB). A 60-second wait already gives 107 tokens/s at 109 GB, and 600 s gives 108 tokens/s at 45 GB. Throughput is nearly flat beyond a minute while traffic keeps falling, so an operator can choose the point that suits the volunteers' data plans.

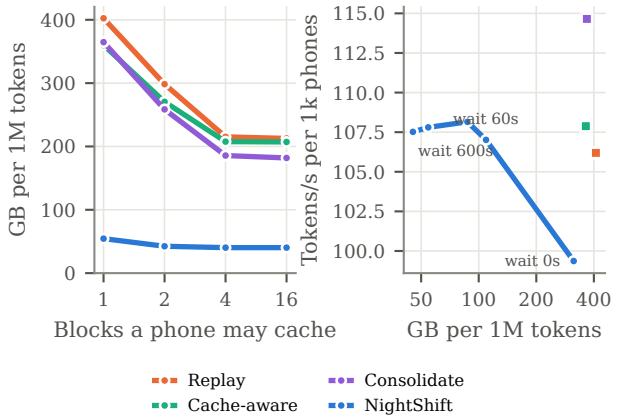


Figure 3: Left: traffic when each phone may cache more blocks. Right: throughput against traffic; NightShift's wait time (0–600 s) traces a trade-off, squares are baselines.

NightShift gets better with scale. With more phones, each block has more idle owners to choose from, so a vacant stage rarely waits long or falls back to a download (Figure 4). From 500 to 4,000 phones, NightShift's throughput rises from 98 to 127 tokens/s per 1,000 phones and its traffic falls from 162 to 35 GB per million tokens, while the baselines stay roughly flat (replay 108 to 109, consolidation 115 to 119). At 500 phones NightShift is worse: -9% against replay and -14% against consolidation, with only 2× less traffic. At 4,000 it is +17% against replay and +7% against consolidation, with 12× less traffic than replay. A deployment worth building would have far more than 4,000 phones, but we have not simulated beyond that, and a large pool would likely be split by region (below), which brings each region back toward smaller pools.

Sensitivity. Figure 5 varies one assumption at a time. Three matter a lot. The share of nights a phone charges moves NightShift's

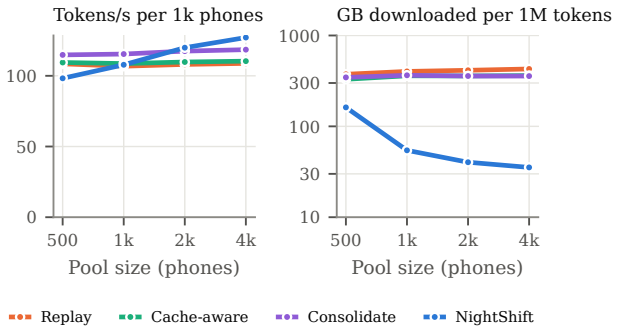


Figure 4: NightShift improves with pool size; the baselines do not. Traffic (shards plus activations) is on a logarithmic scale.

throughput from 58 to 138 tokens/s per 1,000 phones between 50% and 100% of nights (at 50%, replay is slightly ahead). The batch size moves it from 61 (16 sequences) to 146 (48), because more sequences keep more stages busy; memory for the KV cache caps this. The cost ratio of decode to prefill moves it from 58 (ratio 2) to 134 (ratio 32): the prompt of every batch is processed once per stage, and each repair recomputes the cache of all tokens so far. Longer daytime sessions help only modestly (105 to 121), confirming that the night carries the system. Hop latency sets per-token latency (Figure 6: 2.0 s at 5 ms, 2.3 s at 60 ms, 3.2 s at 120 ms) and, because a round cannot be shorter than one token's trip, also throughput (120 to 76). Grouping pipelines by region, so that hops stay short, is therefore worth more than any scheduling refinement. Across every setting we tried, NightShift downloads 2–12× less than replay, the low end at 500 phones; its throughput stays within a few percent of replay's except at 500 phones and at 50% of nights, where it is behind.

7 TOWARD VALIDATION

Simulation with assumed parameters can show which design choices matter, but not absolute numbers. We release a benchmark kit that replaces each assumption with a measurement and reruns every experiment unchanged:

- **Compute.** llama.cpp [10] on Android (Termux) measures decode and prefill speed for an 8B model, the cost of batching 1–8 sequences, sustained speed under thermal throttling, and battery draw. Per-layer time is scaled to the 70B layer by bytes of weights read, since phone decode is memory-bandwidth bound.
- **Network.** Round-trip time and throughput between phones on the same and on different networks.
- **Pipeline.** llama.cpp's RPC backend runs an 8B model split over 1–4 phones, validating the latency model (sum of stage times plus hops).
- **Availability.** A logger records charging and Wi-Fi state every five minutes for 14 days on volunteers' phones; the trace replaces the synthetic diurnal model. Volunteers give informed consent, the logger records no content, location or identifiers beyond a random ID, and it can be removed at any time. Large published traces [29] are an alternative.

This version reports simulation results only. A later version will replace Table 1 with the measured values and repeat every experiment with them.

8 LIMITATIONS AND ETHICS

Privacy. A stage sees the activations of the prompts that pass through it, and activations can leak their input. We therefore restrict the design to work whose inputs are not sensitive (public corpora, synthetic data) until a defence exists.

Integrity. A volunteer can return wrong activations. Redundant execution [2] doubles cost; lightweight checks such as TOPLOC [22] are promising and untested on phones.

Owners. Overnight compute adds heat and battery wear and uses the owner's electricity. Participation must be explicit, revocable and limited by temperature; the federated learning eligibility rule is a minimum, not a licence. We have not modelled incentives.

Model fidelity. The simulator ignores stragglers within a stage, thermal throttling over a night, Wi-Fi contention in one home, and iOS's stricter background limits. Downloads are counted and delay the stage that waits for them, but do not compete with other traffic on a link. Time advances in 10-second ticks, which coarsens short events. Each result comes from one synthetic 24-hour window per seed after warm-up, from a synthetic availability model; real traces may have more correlated departures (for example, everyone unplugging at 07:00 on a weekday) than ours.

9 CONCLUSION

A pool of charging phones can, in principle, run a model far larger than any phone, but only as an overnight batch system with seconds per output token and far more memory traffic per token than a GPU. Within that regime, the cost that matters most to the volunteers is data: replacing leavers moves model shards, and even a cache-aware design moves several gigabytes per phone per day. Giving each phone a permanent home block, waiting briefly for a phone that holds it, and consolidating broken pipelines cut that traffic 6–7× in our simulation, at a few percent of throughput at 1,000 phones and a gain at 4,000; standbys hurt. These results rest on assumed parameters. The next step is to replace them with measurements from real phones and real owners.

REFERENCES

[1] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics.

[2] David P. Anderson. 2004. BOINC: A System for Public-Resource Computing and Storage. In *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing (GRID)*. IEEE, 4–10.

[3] Mustafa Y. Arslan, Indrajeet Singh, Shailendra Singh, Harsha V. Madhyastha, Karthikeyan Sundaresan, and Srikanth V. Krishnamurthy. 2012. Computing While Charging: Building a Distributed Computing Infrastructure Using Smartphones. In *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. ACM.

[4] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage,

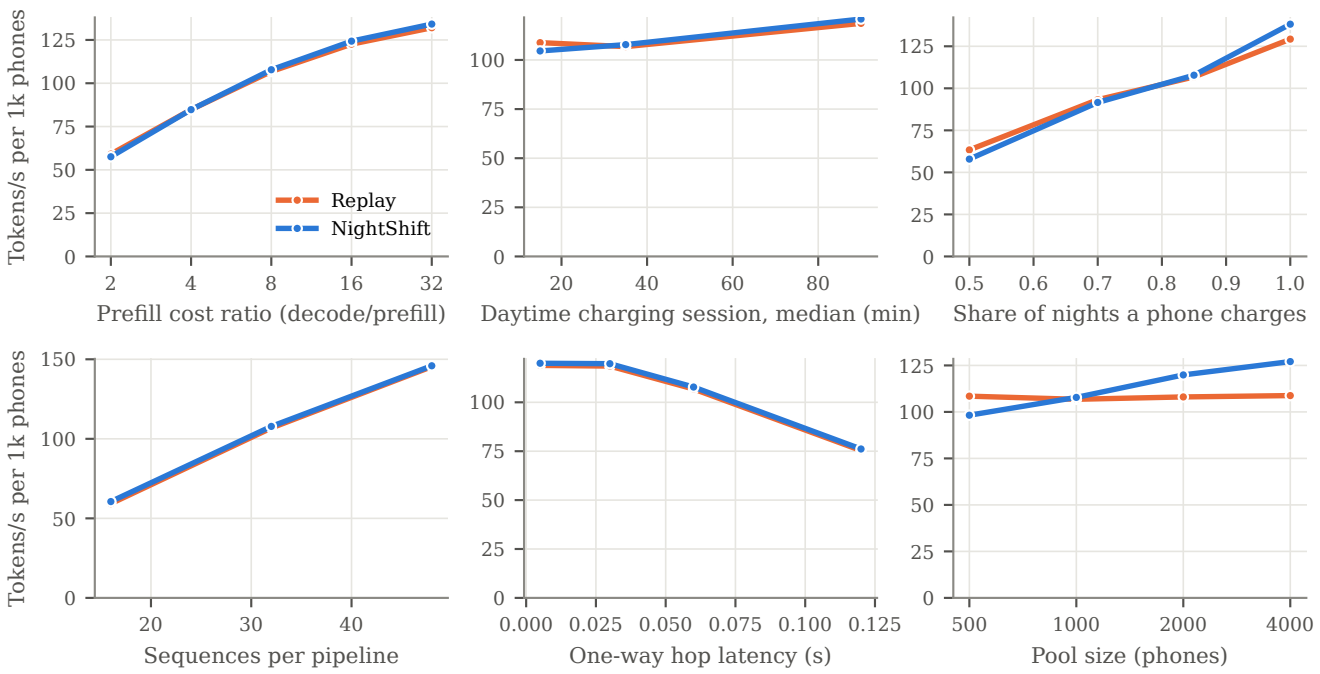


Figure 5: Sensitivity of throughput to the main assumptions, replay vs. NightShift (1,000 phones, mean of 3 seeds).

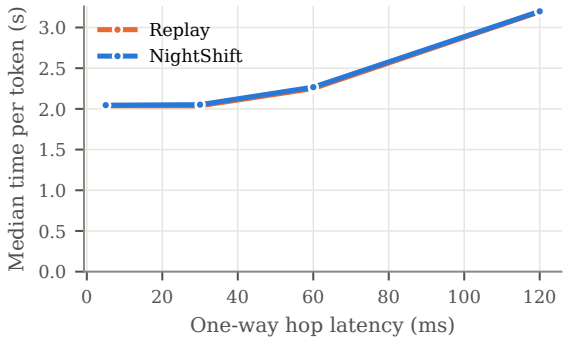


Figure 6: Median time per output token against one-way hop latency.

and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *Proceedings of Machine Learning and Systems (MLSys)*. mlsys.org. arXiv:1902.01046.

[5] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Ryabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative Inference and Fine-tuning of Large Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*. Association for Computational Linguistics.

[6] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin Raffel. 2023. Distributed Inference and Fine-tuning of Large Language Models Over The Internet. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates.

[7] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. 2023. DiLoCo: Distributed Low-Communication Training of Language Models. arXiv:2311.08105.

[8] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. 2024. Parcae: Proactive, Liveput-Optimized DNN Training on Preemptible Instances. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 1121–1139.

[9] exo labs. 2024. exo: Run your own AI cluster at home with everyday devices. https://github.com/exo-explore/exo.

[10] Georgi Gerganov and contributors. 2023. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp.

[11] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates.

[12] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. 2023. Oobleck: Resilient Distributed Training of Large Models Using Pipeline Templates. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM. https://doi.org/10.1145/3600006.3613152

[13] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen, and Binhang Yuan. 2023. HexGen: Generative Inference of Large Language Model over Heterogeneous Environment. arXiv:2311.11514. arXiv:2311.11514

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. ACM.

[15] Fan Lai, Yinwei Dai, Sanjay S. Singapuram, Jiachen Liu, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury. 2022. FedScale: Benchmarking Model and System Performance of Federated Learning at Scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*. PMLR.

[16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR.

[17] Zonghang Li, Tao Li, Wenjiao Feng, Rongxing Xiao, Jianshu She, Hong Huang, Mohsen Guizani, Hongfang Yu, Qirong Ho, Wei Xiang, and Xue Liu. 2026. Prima.cpp: Fast 30–70B LLM Inference on Heterogeneous and Low-Resource Home Clusters. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.

[18] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024. MobileLLM: Optimizing Sub-billion

Parameter Language Models for On-Device Use Cases. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR.

[19] Jiachen Mao, Xiang Chen, Kent W. Nixon, Christopher Krieger, and Yiran Chen. 2017. MoDNN: Local Distributed Mobile Computing System for Deep Neural Network. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE.

[20] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR.

[21] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. 2024. SpotServe: Serving Generative Large Language Models on Preemptible Instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM.

[22] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. 2025. TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. PMLR, 47196–47211.

[23] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. 2023. SWARM Parallelism: Training Large Models Can Be Surprisingly Communication-Efficient. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR.

[24] Max Ryabinin and Anton Gusev. 2020. Towards Crowdsourced Training of Large Neural Networks using Decentralized Mixture-of-Experts. In *Advances in Neural Information Processing Systems (NeurIPS)*. Curran Associates.

[25] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR.

[26] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association.

[27] Linyu Wu, Xiaoyuan Liu, Tianneng Shi, Zhe Ye, and Dawn Song. 2025. DeServe: Towards Affordable Offline LLM Inference via Decentralization. arXiv:2501.14784. arXiv:2501.14784

[28] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. arXiv:2406.06282.

[29] Chengxu Yang, Qipeng Wang, Mengwei Xu, Zhenpeng Chen, Kaigui Bian, Yunxin Liu, and Xuanzhe Liu. 2021. Characterizing Impacts of Heterogeneity in Federated Learning upon Large-Scale Smartphone Data. In *Proceedings of the Web Conference (WWW)*. ACM.

[30] Shengyuan Ye, Jiangsu Du, Liekang Zeng, Wenzhong Ou, Xiaowen Chu, Yutong Lu, and Xu Chen. 2024. Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. In *Proceedings of IEEE INFOCOM*. IEEE. arXiv:2405.17245.

[31] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association.

[32] Mingjin Zhang, Jiannong Cao, Xiaoming Shen, and Zeyang Cui. 2024. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. arXiv:2405.14371.

[33] Junchen Zhao, Yurun Song, Simeng Liu, Ian G. Harris, and Sangeetha Abdu Jyothi. 2023. LinguaLinked: A Distributed Large Language Model Inference System for Mobile Devices. arXiv:2312.00388.

[34] Zhuoran Zhao, Kamyar Mirzazad Barijough, and Andreas Gerstlauer. 2018. DeepThings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018).